

# Clase 2 — Datos en microservicios

---

- Clase 2 — Datos en microservicios
  - 1. Introducción
  - 2. El problema del shared database
  - 3. Database per Service
  - 4. Extraer y transferir el ownership de los datos
  - 5. Transacciones distribuidas: el problema de la consistencia
  - 6. SQL vs NoSQL: criterios de selección
  - 7. MongoDB en profundidad
  - 8. Trade-offs y decisiones arquitectónicas
  - 9. Puntos clave
  - 10. Referencias y profundización

# Clase 2 — Datos en microservicios

---

ARQ 2026 · UCC

---

## 1. Introducción

---

Una de las decisiones más importantes al diseñar microservicios es cómo cada servicio gestiona sus datos. Esta decisión determina el grado de autonomía real de cada servicio, la posibilidad de escalar independientemente, la consistencia del sistema y la libertad para elegir tecnologías adecuadas a cada problema.

Separar los servicios en términos de código y deploy no es suficiente si todos comparten la misma base de datos. El aislamiento real de un microservicio requiere que sus datos sean privados: solo accesibles a través de su propia interfaz.

Este apunte cubre tres problemas centrales:

1. **Propiedad de los datos:** el principio de *Database per Service* y cómo transferir el ownership durante una migración.
  2. **Consistencia:** qué se pierde al distribuir datos y qué patrones permiten gestionar esa pérdida.
  3. **Elección de tecnología:** criterios para seleccionar la base de datos adecuada para cada servicio, con foco en MongoDB como alternativa documental.
-

## 2. El problema del shared database

---

### 2.1 Por qué el modelo compartido funciona en el monolito

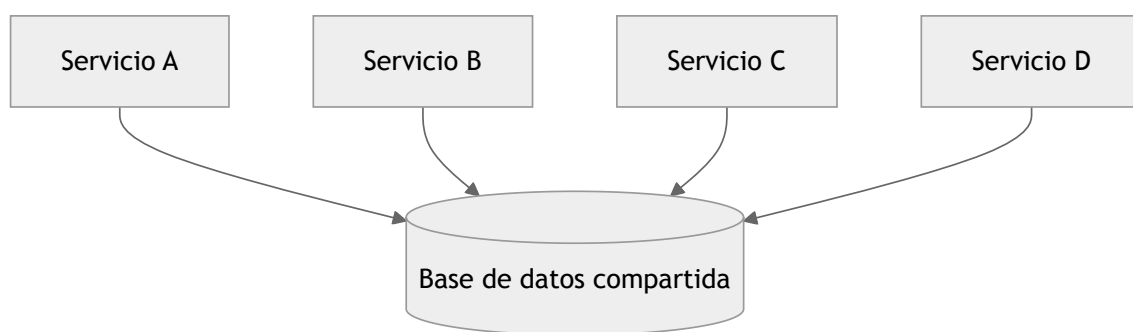
En una arquitectura monolítica, toda la lógica de negocio comparte una única base de datos. Esto tiene ventajas reales:

- **Transacciones ACID** entre entidades del mismo dominio.
- **JOINS directos** entre tablas sin overhead de red.
- **Un único esquema coherente** como fuente de verdad.
- **Simplicidad operacional**: una sola base de datos para mantener.

Cuando el sistema es pequeño y el equipo es uno, este modelo funciona bien.

### 2.2 El antipatrón: extraer servicios sin separar datos

El error más común al migrar a microservicios es separar el código en servicios pero mantener una única base de datos compartida. El resultado es:



A primera vista parece una transición válida, pero los problemas del monolito no desaparecen — solo se camuflan.

**Acoplamiento estructural**: un servicio puede acceder directamente a las tablas internas de otro. Si un equipo cambia el esquema de sus tablas, puede romper silenciosamente a otros servicios que leen esas mismas tablas.

**Dependencia de deploy**: una migración de esquema en las tablas de un servicio puede afectar a otros servicios que leen esas mismas tablas. Un cambio simple se convierte en una operación coordinada entre múltiples equipos — lo opuesto al deploy independiente que prometían los microservicios.

**Imposibilidad de escalar datos de forma independiente**: si un servicio necesita más capacidad de escritura, no puede escalar solo su base sin afectar a todos los demás.

**Lock-in tecnológico**: todos los servicios deben usar el mismo motor de base de datos, aunque distintos problemas se resuelvan mejor con tecnologías diferentes.

La Shared Database es en muchos casos un antipatrón en arquitecturas de microservicios. Puede ser una etapa transitoria válida durante una migración incremental, pero si se mantiene como diseño permanente, reaparecen los problemas que se querían resolver.

---

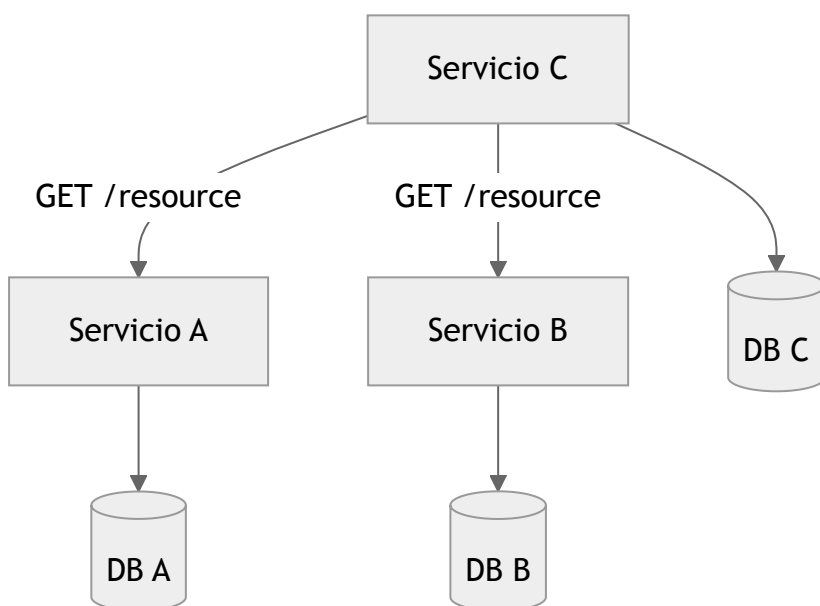
## 3. Database per Service

---

### 3.1 El principio fundamental

**Database per Service** establece que cada microservicio es el único dueño de sus datos. Las tablas, colecciones o cualquier estructura de almacenamiento son un detalle de implementación privado — invisible e inaccesible para cualquier otro servicio.

La única forma de acceder a los datos de un servicio desde afuera es: 1. **Llamar a su API** — el servicio procesa la consulta y devuelve lo que considera apropiado compartir. 2. **Consumir sus eventos** — el servicio publica eventos que otros servicios consumen de forma asíncrona.



### 3.2 Variantes del patrón

Database per Service no implica necesariamente provisionar una instancia física de base de datos por servicio. Hay variantes con distintos niveles de aislamiento:

| Variante           | Descripción                                 | Aislamiento   | Cuándo usarla                                     |
|--------------------|---------------------------------------------|---------------|---------------------------------------------------|
| Tabla separada     | Tablas propias dentro de la misma instancia | Lógico        | Etapas tempranas de migración, baja escala        |
| Schema separado    | Schema propio dentro de la misma instancia  | Lógico fuerte | Seguridad de acceso sin overhead operacional      |
| Instancia separada | Base de datos completamente independiente   | Físico        | Alta escala, compliance estricto, equipos maduros |

La variante apropiada depende del contexto: escala del sistema, requisitos de compliance, costo operacional y madurez del equipo. No toda instancia productiva requiere una base de datos física propia.

### 3.3 Beneficios concretos

**Desacoplamiento real:** un servicio puede cambiar completamente su schema interno sin afectar a ningún otro servicio. Los consumidores acceden a través de la API, que puede mantenerse estable mientras el schema interno evoluciona.

**Libertad tecnológica:** cada servicio puede usar la base de datos más adecuada para su caso de uso. Un servicio puede usar MongoDB para documentos jerárquicos; otro puede usar PostgreSQL para garantías transaccionales.

**Escalado independiente:** si un servicio recibe más tráfico, se puede escalar solo esa base de datos sin afectar las demás.

**Deploy independiente:** los cambios de schema de un servicio no requieren coordinación con otros equipos.

### 3.4 Costos y consecuencias

**Se pierde el JOIN entre entidades de distintos servicios:** lo que antes era una query con un JOIN ahora requiere llamadas a servicios distintos.

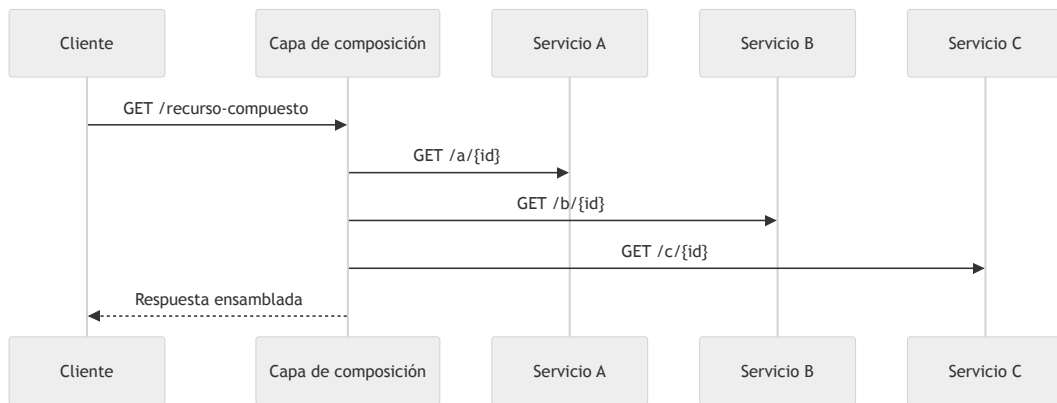
**Los datos necesarios para una respuesta pueden estar en múltiples servicios:** mostrar información compuesta requiere múltiples llamadas coordinadas.

**Referencias sin integridad referencial:** en una base de datos relacional, una foreign key garantiza que no puede existir un registro que referencie un ID inexistente. En microservicios, esa garantía desaparece. Los IDs externos se conservan como campos, pero la validación es responsabilidad del servicio dueño.

### 3.5 Patterns para componer datos entre servicios

Cuando un caller necesita datos que viven en múltiples servicios, hay dos enfoques principales:

**API Composition:** un servicio (o una capa de composición intermedia) llama a varios servicios y ensambla la respuesta antes de devolvérsela al cliente. Es el patrón más simple y suficiente para la mayoría de los casos.



**CQRS** (Command Query Responsibility Segregation): separar las operaciones de escritura (commands) de las de lectura (queries). Las lecturas pueden usar vistas materializadas que desnormalizan datos de múltiples servicios para responder queries complejas de forma eficiente. Permite optimizar el modelo de lectura independientemente del modelo de escritura.

En un sistema distribuido, esas vistas suelen actualizarse a partir de eventos publicados por los servicios dueños. Por eso su información puede tener un retraso: CQRS optimiza consultas complejas a cambio de aceptar consistencia eventual en el modelo de lectura.

---

## 4. Extraer y transferir el ownership de los datos

---

### 4.1 El problema de la transición

Cuando se extrae un servicio de un monolito, sus datos siguen en la base compartida. Hay un período en que coexisten: - El monolito todavía escribe registros (para el tráfico que todavía maneja). - El nuevo servicio también escribe registros (para el tráfico que ya migró).

Si ambos escriben a la vez sin coordinación, los datos divergen.

### 4.2 Estrategia gradual

La migración se puede ejecutar en fases:

**Fase 1 — Expand (copiar sin cortar):** copiar los datos al almacenamiento del nuevo servicio. El servicio comienza a manejar tráfico de lectura; las escrituras todavía van al sistema original y se sincronizan.

**Fase 2 — Coexistencia controlada:** el nuevo servicio maneja escrituras para el tráfico migrado. Debe definirse con precisión quién es el escritor autorizado para cada registro o flujo; dos sistemas no deben escribir el mismo dato sin un mecanismo explícito de sincronización. Los contratos de datos son compatibles con ambos sistemas.

**Fase 3 — Corte de ownership:** se define el momento exacto en que el nuevo servicio es el único dueño de sus datos. El sistema original deja de escribir directamente. Todo acceso pasa por la API del nuevo servicio.

**Fase 4 — Contract (limpiar):** se eliminan las estructuras de transición (tablas duplicadas, sincronizadores, adapters).

La clave es que el corte de ownership sea una decisión explícita con un momento definido — no un estado en el que ambos sistemas coexisten indefinidamente.

---

## 5. Transacciones distribuidas: el problema de la consistencia

---

### 5.1 La garantía que perdemos

En una arquitectura monolítica con una sola base de datos, una transacción SQL garantiza **ACID**:

- **Atomicidad:** todos los cambios ocurren juntos o ninguno ocurre.
- **Consistencia:** el sistema pasa de un estado válido a otro estado válido.
- **Aislamiento:** las transacciones concurrentes no se interfieren entre sí.
- **Durabilidad:** los cambios comprometidos persisten ante fallos.

Cuando los datos de un proceso de negocio están distribuidos en múltiples servicios con bases de datos independientes, ya no existe una transacción local que cubra todo el proceso. Existen coordinadores distribuidos como *two-phase commit* (2PC/XA), pero añaden acoplamiento, latencia y problemas de disponibilidad; por eso no suelen ser la opción por defecto en microservicios.

### 5.2 Consistencia eventual vs. consistencia fuerte

**Consistencia fuerte:** todos los lectores ven el mismo estado al mismo tiempo. Es el modelo al que estamos acostumbrados con bases de datos transaccionales. Requiere coordinación entre nodos, lo que tiene costo en latencia y disponibilidad.

**Consistencia eventual:** el sistema garantiza que, dado tiempo suficiente sin nuevas escrituras, todos los lectores convergerán al mismo estado. Los estados intermedios pueden ser inconsistentes. Este modelo es más escalable pero requiere que la aplicación tolere la inconsistencia transitoria.

La mayoría de los sistemas de microservicios aceptan consistencia eventual para muchas de sus operaciones. El desafío de diseño es identificar cuáles operaciones requieren consistencia fuerte y cuáles pueden tolerar eventual.

### 5.3 El patrón Saga

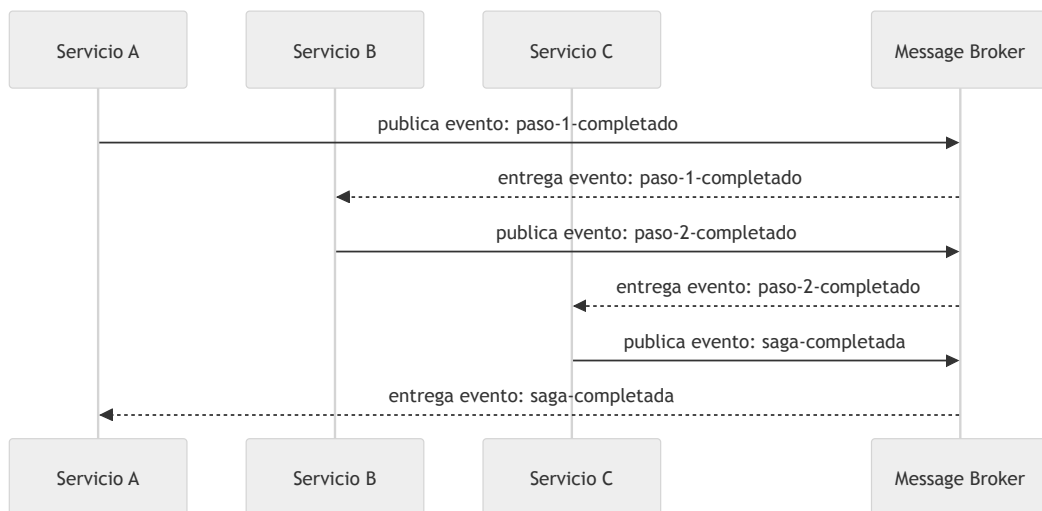
Una **Saga** es una secuencia de transacciones locales — una por servicio involucrado — donde cada transacción local actualiza los datos de su propio servicio y desencadena la siguiente transacción en la secuencia.

Si un paso falla, la Saga ejecuta **transacciones compensatorias** para los pasos anteriores que ya tuvieron éxito. Es importante entender qué son estas compensaciones:

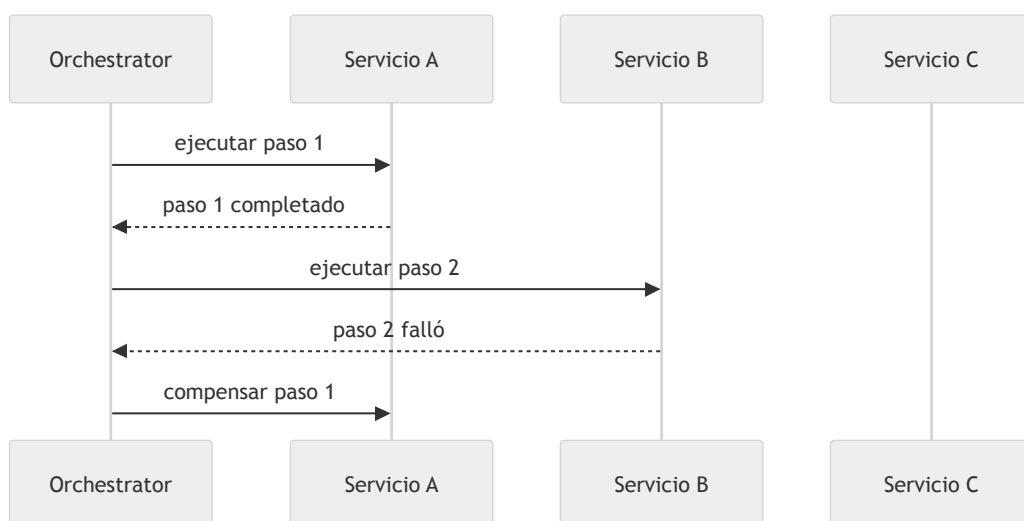
- **No son rollbacks:** un rollback deshace una transacción en la misma base de datos, atómicamente. Las compensaciones son nuevas operaciones de negocio que mitigan los efectos anteriores.
- **No garantizan restaurar el estado exactamente:** si entre el paso exitoso y la compensación llegaron otras operaciones, el estado no es idéntico al original.
- **Son operaciones de negocio reales:** devolver un stock, anular un cobro, cancelar una reserva.

### Variantes del patrón Saga

**Coreografía:** cada servicio reacciona a eventos publicados por el servicio anterior. No hay un coordinador central. Es más simple de implementar pero más difícil de razonar sobre el flujo completo y de depurar cuando algo falla.



**Orquestación:** un servicio coordinador (o proceso dedicado llamado *saga orchestrator*) dirige explícitamente los pasos de la saga y maneja las compensaciones cuando algo falla. Es más complejo de implementar pero más fácil de observar, depurar y modificar.



## 5.4 El problema del dual write

Cuando un servicio necesita tanto guardar un cambio en su base de datos como publicar un evento que notifique a otros servicios, enfrenta el **problema del dual write**: las dos acciones son independientes y cualquiera puede fallar:

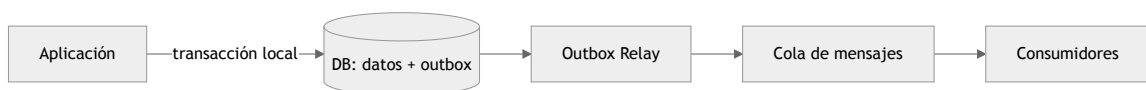
```
// El problema
guardarCambio(db, datos) // puede fallar
publicarEvento(cola, evento) // puede fallar independientemente
// Si la aplicación muere entre las dos operaciones,
// el estado de la base y el de la cola divergen.
```

No existe forma de hacer atómica una escritura en base de datos y una publicación en una cola de mensajes sin un mecanismo específico.

## 5.5 El patrón Transactional Outbox

La solución al dual write es el patrón **Transactional Outbox**:

En lugar de publicar directamente a la cola, el servicio escribe el evento pendiente en una tabla de la misma base de datos (*outbox*), dentro de la misma transacción local que guarda el cambio de negocio. Un proceso separado (el *outbox relay*) lee los eventos pendientes y los publica a la cola.



```
-- Todo dentro de una sola transacción local:
INSERT INTO pedidos (id, usuario_id, monto, estado) VALUES (...)
INSERT INTO outbox (id, tipo_evento, payload, creado_en)
  VALUES (uuid(), 'pedido.creado', '{...}', now())
-- Si la transacción falla: no hay pedido NI evento pendiente.
-- Si tiene éxito: el relay publicará el evento eventualmente.
```

**Qué garantiza el patrón:**

- El cambio de negocio y la intención de publicar su evento se confirman juntos en una única transacción local.
- Si el relay falla antes de publicar, puede reintentar desde el outbox; para que el evento no se pierda, el outbox debe retenerse y monitorearse correctamente.
- El relay puede publicar el mismo evento más de una vez si falla después de publicar pero antes de marcarlo como procesado. Por eso la entrega suele ser *at-least-once*, no *exactly-once*.

## 5.6 Idempotencia

Dado que en sistemas distribuidos los mensajes pueden llegar más de una vez (por reintentos del relay, por la semántica *at-least-once* de la mayoría de las colas), los consumidores deben ser **idempotentes**: procesar el mismo mensaje múltiples veces debe producir el mismo resultado que procesarlo una sola vez.

Una implementación común es mantener un registro de los IDs de mensajes procesados. El registro y el efecto de negocio deben confirmarse en la **misma transacción local**: marcar un mensaje como procesado antes de completar su efecto produciría una pérdida lógica si el proceso falla luego.

```
CREATE TABLE mensajes_procesados (  
  mensaje_id UUID PRIMARY KEY,  
  procesado_en TIMESTAMPTZ  
);  
  
-- Todo dentro de una misma transacción local:  
INSERT INTO mensajes_procesados (mensaje_id, procesado_en)  
VALUES (:mensaje_id, now())  
ON CONFLICT (mensaje_id) DO NOTHING  
RETURNING mensaje_id;  
  
-- Si no se devuelve una fila, el mensaje ya fue procesado: no hacer nada más.  
-- Si se devuelve una fila, aplicar el cambio de negocio y confirmar ambas  
-- escrituras juntas en esta misma transacción local.
```

La idempotencia no es una optimización opcional — es un requerimiento de diseño en sistemas distribuidos que priorizan la entrega garantizada de mensajes.

## 5.7 El trade-off fundamental

Las transacciones distribuidas son inherentemente más complejas y costosas que las transacciones locales. Una estrategia efectiva para reducir su necesidad es el **diseño de dominio cuidadoso**: poner todo lo que debe ser atómico dentro del mismo servicio.

Si varias operaciones deben ocurrir juntas o no ocurrir, puede ser una señal de que pertenecen al mismo servicio — no a servicios distintos. La separación en servicios no debe sacrificar la coherencia del modelo de negocio.

---

## 6. SQL vs NoSQL: criterios de selección

---

### 6.1 Bases de datos relacionales (SQL)

El modelo relacional organiza los datos en tablas con filas y columnas. Las relaciones entre entidades se expresan mediante foreign keys. Las garantías ACID están incorporadas en el motor.

**Cuándo son la elección correcta:** - Los datos tienen relaciones complejas y bien definidas que se expresan naturalmente en el modelo relacional. - Se necesitan JOINS frecuentes entre múltiples entidades. - Los requerimientos de consistencia transaccional son estrictos (finanzas, reservas, salud). - El equipo tiene experiencia con el modelo relacional. - Hay requisitos de auditoría o compliance que se benefician de la madurez del ecosistema relacional.

**Motores representativos:** PostgreSQL, MySQL, SQLite.

Un error frecuente es descartar las bases relacionales prematuramente. Un PostgreSQL bien configurado — con connection pooling, réplicas de lectura y una capa de caché — puede manejar cargas muy altas. La complejidad operacional de una base distribuida NoSQL es un costo real que debe justificarse.

## 6.2 Familias de bases de datos NoSQL

El término “NoSQL” agrupa tecnologías con características muy distintas. Las principales familias son:

### Documentales

**Ejemplos:** MongoDB, CouchDB, Firestore

**Modelo:** almacenan documentos JSON que pueden contener subdocumentos y arrays. Dos documentos en la misma colección pueden tener estructuras diferentes.

**Cuándo usar:** datos jerárquicos o semiestructurados, esquema que varía entre instancias, entidades que se leen frecuentemente como unidad completa.

### Clave-valor

**Ejemplos:** Redis, DynamoDB, Memcached

**Modelo:** el acceso principal se realiza por clave. Algunos productos ofrecen estructuras más ricas o índices secundarios, pero el diseño debe partir de patrones de consulta conocidos y acotados.

**Cuándo usar:** caché, sesiones de usuario, configuraciones, contadores. El patrón de acceso debe reducirse a “dame el valor para esta clave”.

### Columnares

**Ejemplos:** Apache Cassandra, HBase

**Modelo:** son almacenes de columnas anchas (*wide-column*), organizados alrededor de una clave de partición y patrones de consulta definidos de antemano. No son equivalentes a los almacenes analíticos columnares.

**Cuándo usar:** escritura masiva con alta disponibilidad, patrones de lectura predecibles, series de tiempo y telemetría.

Los almacenes analíticos columnares, como Amazon Redshift, BigQuery o ClickHouse, son otra categoría: optimizan agregaciones sobre grandes volúmenes y no suelen ser la base transaccional de un microservicio.

### Grafos

**Ejemplos:** Neo4j, Amazon Neptune

**Modelo:** nodos y relaciones son ciudadanos de primera clase. Las consultas navegan el grafo de forma nativa.

**Cuándo usar:** redes sociales, motores de recomendación, detección de fraude basada en patrones de conexión, rutas.

## Series de tiempo

**Ejemplos:** InfluxDB, Prometheus, TimescaleDB

**Modelo:** los datos se organizan principalmente por timestamp. Están optimizadas para escrituras continuas y consultas de rango temporal. TimescaleDB es una extensión relacional de PostgreSQL, no una base NoSQL.

**Cuándo usar:** métricas de aplicaciones, telemetría de dispositivos IoT, precios históricos.

## 6.3 Criterios de selección

Elegir una base de datos es una decisión arquitectónica con consecuencias a largo plazo. Los criterios relevantes son:

**Estructura del dato:** ¿el esquema es estable y bien definido, o varía entre instancias o evoluciona rápido? Un esquema estable favorece SQL; uno flexible favorece documentales.

**Tipo de queries:** ¿son JOINS entre entidades relacionadas, lookups por clave, búsquedas full-text, o agregaciones temporales? Las queries determinan qué motor puede servirlos eficientemente.

**Patrón de acceso:** ¿el sistema es read-heavy o write-heavy? ¿Las queries son predecibles o ad-hoc? Las bases documentales son más eficientes cuando se lee la entidad completa. Las columnares cuando se necesitan columnas específicas de muchas filas.

**Escala:** ¿el volumen requiere escalar horizontalmente? Algunas bases relacionales escalan horizontalmente, pero la mayoría de las NoSQL fue diseñada con eso como prioridad.

**Consistencia requerida:** ¿el caso de negocio tolera consistencia eventual, o requiere ACID estricto? Las operaciones financieras generalmente requieren ACID; los feeds de actividad pueden tolerar eventual.

**Madurez del equipo:** ¿el equipo conoce la tecnología? ¿Hay operadores disponibles? Una base técnicamente superior que nadie sabe operar no es una buena elección.

---

## 7. MongoDB en profundidad

---

### 7.1 El modelo de datos documental

La unidad de almacenamiento en MongoDB es el **documento**: un objeto JSON (almacenado internamente en formato BSON — Binary JSON) que puede contener campos simples, arrays y subdocumentos anidados.

```
{
  "_id": ObjectId("64a7f2c3e1b2d3a4b5c6d7e8"),
  "nombre": "Cosquín Rock 2027",
  "fecha": "2027-02-14",
  "venue": {
    "nombre": "Anfiteatro Próspero Molina",
    "ciudad": "Cosquín",
    "capacidad": 15000
  },
  "artistas": [
    { "nombre": "Divididos", "horario": "20:00", "escenario": "principal" },
    { "nombre": "Los Piojos Reunion", "horario": "22:00", "escenario": "principal" }
  ],
  "categorias": ["rock", "nacional"],
  "precio_base": 8500,
  "estado": "publicado"
}
```

Los documentos se agrupan en **colecciones** — análogo a las tablas en SQL, pero sin esquema impuesto. El campo `_id` es el identificador único del documento. Por defecto, MongoDB genera un `ObjectId` que incluye un timestamp y un componente aleatorio — globalmente único sin coordinación central.

## 7.2 Embedding vs. references

La decisión de modelado más importante en MongoDB es cuándo **embeber** subdocumentos (incluirlos dentro del documento padre) y cuándo usar **referencias** (guardar solo el ID del documento relacionado).

**Embedding:** los datos del subdocumento se guardan dentro del documento padre.

```
{
  "_id": ObjectId("..."),
  "nombre": "Cosquín Rock 2027",
  "artistas": [
    { "nombre": "Divididos", "bio": "Banda de rock argentino formada en 1988..." },
    { "nombre": "Los Piojos Reunion", "bio": "..." }
  ]
}
```

✅ Usar cuando el subdocumento no tiene vida propia fuera del padre, se lee casi siempre junto al padre, y la colección de subdocumentos no crece indefinidamente.

**References:** el documento guarda solo el ID del documento relacionado.

```
{
  "_id": ObjectId("..."),
  "event_id": ObjectId("64a7f2c3..."),
  "user_id": ObjectId("98b4c1d2..."),
  "precio": 8500,
  "estado": "pagado"
}
```

✓ Usar cuando la entidad relacionada tiene ciclo de vida independiente, puede ser compartida entre muchos documentos, o crece en tamaño de forma no acotada.

La regla general es elegir según los **patrones de acceso**: si siempre se lee la entidad completa, embeber. Si la entidad relacionada se consulta frecuentemente de forma independiente, referenciar.

### 7.3 Beneficios frente al modelo relacional

**Representación natural de entidades jerárquicas:** una entidad con múltiples subelementos se almacena como un solo documento. En SQL, el mismo modelo puede requerir varias tablas con JOINS.

Por ejemplo, una entidad con venue, artistas y categorías en SQL:

```
CREATE TABLE events (id, name, price, status);
CREATE TABLE venues (id, name, city, capacity);
CREATE TABLE event_venues (event_id, venue_id);
CREATE TABLE artists (id, name, bio);
CREATE TABLE event_artists (event_id, artist_id, time, stage);
CREATE TABLE categories (id, name);
CREATE TABLE event_categories (event_id, category_id);
```

Para reconstruir la entidad completa se necesita un JOIN entre siete tablas. En MongoDB, es una sola lectura.

**Menor impedancia de mapeo:** los structs de Go o los objetos de JavaScript suelen mapearse de forma más directa a documentos BSON y requieren menos abstracciones complejas que un ORM relacional tradicional. El driver igualmente serializa y deserializa los datos en tiempo de ejecución.

**Esquema flexible:** permite que distintos tipos de entidades tengan campos adicionales sin alterar el schema de todos los demás. Un tipo de entidad puede tener `lista_artistas`; otro puede tener `elenco`; otro puede tener `tracks`. En SQL, esto requeriría columnas nullable o una tabla de extensión.

**Escalado horizontal nativo (sharding):** MongoDB fue diseñado para distribuir datos en múltiples nodos. El sharding automático distribuye las colecciones entre nodos basándose en una clave configurable.

**Operaciones atómicas a nivel de documento:** una escritura sobre un documento completo, incluyendo sus subdocumentos, es atómica. Esto permite actualizar el estado de una entidad y su historial de cambios en una sola operación.

**Performance en lecturas de entidades completas:** cuando todos los datos de una entidad están en un solo documento, una lectura los recupera todos sin latencia adicional de JOINS.

## 7.4 Cuándo MongoDB gana sobre relacional

- El dato principal tiene estructura jerárquica o semiestructurada (arrays de subelementos, campos opcionales que varían entre instancias).
- La entidad se lee casi siempre entera, no con proyecciones selectivas.
- El esquema evoluciona frecuentemente o varía entre instancias de la misma entidad.
- La carga es principalmente de lectura, con acceso por ID o por campos indexados.
- El sistema necesita escalar horizontalmente más allá de lo que un solo nodo relacional puede manejar.

## 7.5 Contras y riesgos reales

Entender las debilidades reales de MongoDB es tan importante como entender sus fortalezas. Estas no son razones para no usarlo — son restricciones que hay que gestionar conscientemente.

### Joins no como patrón central de diseño

MongoDB ofrece `$lookup` en el aggregation pipeline para cruzar datos entre colecciones. Sin embargo, si se necesita hacerlo frecuentemente entre colecciones grandes, las ventajas del modelo documental se erosionan. En entornos con *sharding*, su costo depende de la clave de partición, la localización de los datos y los índices; no debe asumirse que un `$lookup` distribuido será barato.

En la práctica: si se necesita cruzar datos de dos colecciones con frecuencia, es señal de que podrían modelarse en el mismo documento, requerir una vista de lectura desnormalizada, o que el modelo relacional es más apropiado.

### Consistencia eventual en clusters

En una configuración de alta disponibilidad con **Replica Set** (el modo de producción estándar), MongoDB replica los datos entre nodos de forma asíncrona. Las lecturas dirigidas a nodos secundarios pueden devolver datos no del todo actualizados; hay un lag de replicación variable según la carga.

La aplicación debe elegir explícitamente la preferencia de lectura (*read preference*) y el nivel de consistencia (*read concern*). La primera decide qué nodo puede responder la consulta: por ejemplo, `primary` lee siempre del nodo primario, mientras que `secondary` permite leer desde réplicas y puede devolver datos atrasados.

El *read concern*, en cambio, define qué garantía debe cumplir el dato leído. Por ejemplo: - `local` : permite leer datos confirmados por el nodo que responde, sin exigir que estén confirmados por la mayoría. - `majority` : devuelve datos que ya fueron confirmados por una mayoría de nodos; puede sacrificar latencia por una garantía más fuerte.

Ignorar esto puede llevar a bugs sutiles donde un usuario ve datos desactualizados después de una escritura reciente.

### Esquema flexible ≠ sin diseño

La ausencia de schema enforcement en la base de datos no elimina la necesidad de definir y mantener un schema en la aplicación. Sin disciplina, las colecciones acumulan documentos con estructuras inconsistentes — algunos tienen el campo `fecha`, otros `date`, otros `event_date`. Esto hace el código defensivo y los queries frágiles.

El error clásico: guardar documentos heterogéneos en la misma colección sin un campo discriminador (`tipo`, `kind`) que permita distinguirlos.

La recomendación es usar **schema validation** de MongoDB (JSON Schema) o una librería de validación en la aplicación para hacer cumplir la estructura esperada, sin perder la flexibilidad de evolucionar el schema gradualmente.

### Transacciones multi-documento

MongoDB soporta transacciones ACID desde la versión 4.0. Sin embargo, su uso tiene un costo de performance significativo y requiere configuración explícita. El modelo documental está optimizado para que la mayoría de las operaciones sean sobre un solo documento — cuando se necesitan transacciones frecuentes entre múltiples documentos, el diseño del modelo probablemente necesita revisión.

### Duplicación de datos

Para evitar `$lookup` s frecuentes, es común desnormalizar: guardar información redundante dentro de un documento. La ventaja es que una lectura recupera todo; el problema aparece cuando esa información cambia: todos los documentos que la contenían deben actualizarse.

La alternativa es aceptar datos “históricamente correctos al momento de la escritura” — que a veces es el comportamiento correcto (el nombre de un evento en un ticket refleja cómo se llamaba al comprarlo) y a veces es un bug. Este trade-off debe ser una decisión explícita de diseño, no un accidente.

### Queries ad-hoc sin índices

Un query por un campo sin índice dispara un **collection scan**: MongoDB recorre todos los documentos de la colección uno por uno. En colecciones con millones de documentos, esto es inaceptable en producción.

En cualquier motor, los índices se derivan de los patrones de consulta, no aparecen automáticamente por tener foreign keys o constraints. En MongoDB, la planificación de índices debe ser explícita y parte del proceso de diseño, no una corrección a posteriori.

### Aggregation pipeline compleja

El aggregation pipeline de MongoDB (`$match`, `$group`, `$project`, `$lookup`, etc.) es poderoso pero más difícil de razonar y optimizar que SQL para analítica compleja. Las consultas ad-hoc y los reportes se expresan más naturalmente en SQL.

---

## 8. Trade-offs y decisiones arquitectónicas

### 8.1 El costo real de Database per Service

| Beneficio                          | Costo                                                 |
|------------------------------------|-------------------------------------------------------|
| Desacoplamiento y autonomía        | Pérdida de JOINS y transacciones ACID entre servicios |
| Libertad tecnológica               | Mayor complejidad operacional (múltiples motores)     |
| Escalado independiente             | Sincronización de datos entre servicios               |
| Deploy sin coordinación de schemas | Consistencia eventual en lugar de fuerte              |

### 8.2 SQL vs NoSQL — cuándo usar cada uno

| Criterio            | Relacional (SQL)                                            | Documental (MongoDB)                                           | Clave-valor (Redis)                                       |
|---------------------|-------------------------------------------------------------|----------------------------------------------------------------|-----------------------------------------------------------|
| Estructura del dato | Tabular, estable                                            | Jerárquica, variable                                           | Plana, por clave                                          |
| Queries             | JOINS, ad-hoc                                               | Por entidad, con índices                                       | Solo por clave                                            |
| Consistencia        | Transacciones e integridad referencial maduras              | Configurable: transacciones y lecturas con distintas garantías | Depende del producto y de su configuración de replicación |
| Escalado            | Vertical y también réplicas, particionado o <i>sharding</i> | <i>Sharding</i> incorporado como capacidad del motor           | Horizontal según el producto y el patrón de acceso        |
| Mejor para          | Finanzas, relaciones complejas                              | Entidades jerárquicas                                          | Caché, sesiones                                           |

### 8.3 El error más común: NoSQL por moda

El error más frecuente en la industria es adoptar NoSQL porque “escala mejor” sin analizar si el caso de uso lo justifica. En la práctica:

- Un PostgreSQL con connection pooling, réplicas de lectura y caché puede manejar decenas de miles de requests por segundo.
- La complejidad operacional de un cluster MongoDB o Cassandra es significativamente mayor que la de un PostgreSQL en alta disponibilidad.
- Migrar de NoSQL a relacional (o viceversa) después de que el sistema está en producción es costoso y riesgoso.

La pregunta correcta no es “¿SQL o NoSQL?” sino “¿qué características de esta base de datos resuelven los problemas específicos de este servicio?”

---

## 9. Puntos clave

---

- La **Shared Database** en microservicios preserva el acoplamiento del monolito. Puede ser una etapa transitoria válida, pero no un diseño permanente.
  - **Database per Service**: cada servicio es el único dueño de sus datos, accesibles solo por su API o sus eventos. El nivel de aislamiento (tabla, schema, instancia) depende del contexto.
  - Las **referencias entre servicios** se hacen con IDs lógicos. No hay foreign keys entre bases de datos; la integridad es responsabilidad del servicio dueño.
  - Las transacciones ACID distribuidas existen, pero su coordinación tiene un costo alto y no suele ser la opción por defecto. El patrón **Saga** secuencia transacciones locales con compensaciones cuando algo falla. Las compensaciones no son rollbacks — son nuevas operaciones de negocio.
  - El **dual write** crea inconsistencias entre la base de datos y la cola de mensajes. El patrón **Transactional Outbox** confirma el cambio y la intención de publicar en la misma transacción local; suele entregar eventos *at-least-once*.
  - Los consumidores de mensajes deben ser **idempotentes**: el mismo mensaje puede llegar más de una vez en sistemas distribuidos.
  - La **libertad tecnológica** de Database per Service permite elegir la base adecuada para cada problema: SQL para datos relacionales y transaccionales, documental para entidades jerárquicas, clave-valor para caché.
  - **MongoDB** es una buena elección cuando la entidad es jerárquica, el esquema es variable y la entidad se lee como unidad completa. Sus riesgos principales: falta de diseño de schema, consistencia eventual en réplicas, duplicación al desnormalizar y performance sin índices.
  - La decisión SQL vs NoSQL se toma por estructura del dato, tipo de queries, patrón de acceso, escala, consistencia requerida y madurez del equipo — nunca solo por “escala”.
- 

## 10. Referencias y profundización

---

- [Chris Richardson — Microservices Patterns](#) — gestión de datos y transacciones distribuidas, incluidos Saga y Outbox.
- [Martin Fowler — Database per Service](#) — contexto y motivación del patrón.
- [Designing Data-Intensive Applications — Martin Kleppmann](#) — referencia sobre bases de datos distribuidas, consistencia y trade-offs.
- [MongoDB — Data Modeling Introduction](#) — guía oficial sobre embedding y references.
- [PostgreSQL — Transaction Isolation](#) — niveles de aislamiento y garantías ACID.

